

MODUL 3

FOR LOOP

FOR LOOPS

Python menyediakan pernyataan untuk mengiterasi suatu koleksi. Pernyataan **for** mengeksekusi blok kode yang ditentukan untuk setiap elemen dalam koleksi.

```
for x in [1, 2, 3, 4]:  
    print(x)  
  
for i in range(10):  
    print (i, i*i, i*i*i)
```

Koleksi merupakan object iter() yang dapat diiterasi :

```
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    print(x)
```

Atau dalam bentuk sekuen range:

```
for i in range(10):  
    print (i, i*i, i*i*i)
```

Function zip yang merupakan built-in function Python akan mengambil dua list dan mengembalikan daftar pasangan.

```
>>> zip(["a", "b", "c"], [1, 2, 3])  
[('a', 1), ('b', 2), ('c', 3)]
```

Ini berguna ketika kita ingin melakukan iterasi dari dua list bersama-sama.

```
names = ["a", "b", "c"]  
values = [1, 2, 3]  
for name, value in zip(names, values):  
    print(name, value)
```

Problem Perulangan For-Loops :

1. Buatlah kode program menggunakan perulangan for-loops berikut :
 1. Jumlahkan sejumlah bilangan: $1+2+3+\dots+n$, dengan n merupakan bilangan yang diinputkan oleh user
 2. Jumlahkan sejumlah bilangan ganjil: $1+3+5+\dots+n$, dengan n merupakan bilangan ganjil yang diinputkan oleh user.
3. Jika diberikan kode program sebagai berikut, statement apa yang perlu dituliskan dalam blok for, agar diperoleh hasil sesuai dengan output :

Pemrograman
Arsitektur Komputer
Bahasa Inggris
Pancasila

```
mataKuliah = ["Kalkulus", "Pemrograman", "Statistika", "Arsitektur Komputer",  
              "Bahasa Indonesia", "Bahasa Inggris", "Agama", "Pancasila",  
              "Olahraga"]  
  
for x in mataKuliah:  
    . . .
```

4. Tulis program menggunakan for-loops sehingga menghasilkan output sebagai berikut :

2
4
6
8
10
12
14
16
18
20

5. Cetak daftar dalam urutan terbalik menggunakan for-loop :

```
list1 = [10, 20, 30, 40, 50]
```

dengan output yang diharapkan adalah sebagai berikut :

50
40
30
20
10

MODUL 4

WHILE LOOP

WHILE LOOPS:

Pernyataan **while loop** secara struktural mirip dengan klausa **if**, yang terdiri dari **Boolean_expression** (kondisi loop) dan serangkaian **statement**. Interpreter secara berulang akan mengeksekusi **statement** selama hasil evaluasi dari **Boolean_expression** bernilai **True**. Proses eksekusi akan berulang sampai kondisi bernilai **Boolean_expression** bernilai **False**.

```
x = 5
while x > 0:
    print(x)
    x -= 1

print("STOP")
```

Increment

Cobalah contoh sederhana berikut :

```
i = 1
while i < 6:
    print(i)
    i += 1
```

Dengan menggunakan perintah tersebut, maka dilakukan proses mencetak nilai **i** sebanyak 5 kali.

Decrement

Cobalah beberapa contoh sederhana berikut :

```
x = 5
while x > 0:
    print(x)
    x -= 1

print("STOP")
```

Bagaimana outputnya?

Problem 4.1 :

1. Apa yang terjadi ketika kode berikut dijalankan? Apakah itu akan memberikan kesalahan? Jelaskan alasannya.

```
while True:
    print("HELLO")
```

Bagaimana cara mengatasi hal tersebut, Apa yang perlu ditambahkan dalam kode tersebut?

2. Ubah kode tersebut menjadi bentuk while-loops !

```
x = input('Masukkan sebarang angka :')
x = int(x)
if x == 2:
    print(x)
else:
    x = input('Masukkan sebarang angka :')
```

3. Tulis program yang meminta angka dari user dan memeriksa apakah itu bilangan genap atau tidak menggunakan perulangan while-loops. Jika input dari user adalah bilangan genap, maka cetak GENAP dan perulangan berhenti. Namun jika bilangannya ganjil, cetak GANJIL, dan minta input dari user hingga bilangan yang dimasukkan adalah bilangan GENAP
4. Buatlah kode program menggunakan perulangan while-loops berikut :
 - a. Jumlahkan sejumlah bilangan: $1+2+3+\dots+n$, dengan n merupakan bilangan yang diinputkan oleh user
 - b. Jumlahkan sejumlah bilangan ganjil: $1+3+5+\dots+n$, dengan n merupakan bilangan ganjil yang diinputkan oleh user.
 - c. Buatlah program untuk menghitung jumlah digit dalam suatu bilangan menggunakan perulangan while. Misalnya, angkanya adalah 75869, jadi outputnya harus 5.

MODUL 5

NESTED LOOPS

Nested loops adalah sebuah proses perulangan di dalam perulangan, atau bisa dikombinasikan dengan kontrol seleksi if-else.

```
adj = ["red", "big", "tasty"]
fruits = ["apple", "banana", "cherry"]

for x in adj:
    for y in fruits:
        print(x, y)
```

Bagaimana outputnya?

```
for i in range(1, 6):
    for j in range(1, 6):
        print(i * j, end=' ')
    print()
```

Atau bisa juga kombinasi while dengan if-else

```
number_str = input("Enter an int:")
number = int(number_str)
count = 0
while number > 0:
    if number % 2 == 0:
        number = number // 2
    elif number % 3 == 0:
        number = number // 3
    else:
        number = number - 1
    count = count + 1
print("Count is: ",count)
print("Number is: ",number)
```

Bagaimana outputnya jika number = 100? mengapa?

Jika statement else dihapus apa yang terjadi?

Break-continue Statement

Dalam sebuah proses *loops* kita juga dapat menghentikan proses tersebut dengan menambahkan perintah *break* . Sehingga proses berhenti meskipun kondisi dalam control seleksi masih bernilai *true*.

```
i = 1
while i < 6:
    print(i)
    if i == 3:
        break
    i += 1
```

Bagaimana outputnya?

Selain itu, dalam sebuah proses *while loops* kita juga dapat menghentikan proses perulangan dengan menggunakan perintah *continue*, dimana perintah ini akan dijalankan jika menemukan suatu kondisi tertentu.

```
i = 0
while i < 6:
    i += 1
    if i == 3:
        continue
    print(i)
```

Lakukan beberapa contoh sederhana berikut :

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    print(x)
```

dalam sebuah tipe data string, *for loops* dapat dijalankan untuk melakukan seleksi pada tiap karakternya. Contoh :

```
for x in "banana":
    print(x)
```

selain itu, perintah *break* dan *continue* juga berlaku di dalam perulangan *for*.

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    print(x)
    if x == "banana":
        break
```

Problem Nested Loops :

1. Buatlah kode program menggunakan nested perulangan sehingga menghasilkan output berikut :

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Hint : Gunakan For dalam For loops dengan range()

2. Tulis program untuk menampilkan hanya angka-angka dari daftar yang memenuhi kondisi berikut:

- Bilangan harus habis dibagi lima
- Jika jumlahnya lebih besar dari 150, lewati dan pindah ke nomor berikutnya
- Jika jumlahnya lebih besar dari 500, maka hentikan loop

Angka yang diberikan yaitu :

```
numbers = [12, 75, 150, 180, 145, 525, 50]
```

dan output yang diharapkan adalah:

```
75
150
145
```

6. Tulis program yang digunakan untuk for-loop untuk mencetak pola bilangan terbalik berikut:

```
5 4 3 2 1
4 3 2 1
3 2 1
2 1
1
```